C200 Programming Assignment №6

Professor M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

October 27, 2024

Introduction

Due Date: 11:00 PM, Saturday, November 2, 2024 Please note that both Autograder submission `https://c200.luddy.indiana.edu/` and pushing to GitHub **must be done before the deadline. We do not accept late submissions**.

Instructions

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).

2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.

3. Make sure that the **code does not have infinite loop (that never exits) or an endless recursion (that never completes)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.

4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

5. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Problem 1: Recursion to the Rescue

When we looked at converting numbers to different bases, there was a strong recursive element lurking in the background. In fact, the code to convert into a base and return to the original number is much simpler in recursive form. Implement two functions `cbr,d` that convert base 10 to another base $1 < b < 9$ producing a string encoding the base and then taking the string encoding and base converts to base 10, respectively. The recursive function `cbr(x,b=2)` takes a positive integer 1,2,... and converts it into base `b` (default is base 2). The complementary function `d(x,b=2)` takes a string encoding a number in base `b` (default 2) and converts it into decimal. The following code:

```
1 x = cbr(5,3)
2 print(x,d(x,3))
3 x = cbr(11,2)
4 print(x,bin(11),int(bin(11),2),d(x,2))
```

produces

```
1 12 5
2 1011 0b1011 11 11
```

We can confirm this mathematically. $1 \times 3^1 + 2 \times 3^0 = 3 + 2 = 5_{10}$. For the second value:

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^2 + 1 \times 2^0 \quad (1)$$

$$= 8 + 0 + 2 + 1 = 11_{10} \quad (2)$$

Deliverables for Problem 1

- Complete both functions.
- Do not consider zero, since it's zero everywhere.
- You're only allowed to use // and %.

Problem 2: Maximal Value

In this problem, you're given a list of numbers. The function msi takes a list of numbers and returns a list of three values (in this same order) namely, the interval start, interval stop and the value of the greatest sum. For example if the list is: x = [7, -9, 5, 10, -9, 6, 9, 3, 3, 9] then the greatest interval is from x[2:10] giving 36:

$$sum(x[2,10]) \quad = \quad 5+10-9+6+9+3+3+9 = 36 \tag{3}$$

The function should return a list of values i.e., [2, 10, 36]

```
1 x =  [7, -9, 5, 10, -9, 6, 9, 3, 3, 9]
2 print(msi(x))
```

produces

```
1 [2,10,36]
```

For this problem you are allowed to use sum(lst) which returns the sum of a non-empty list of numbers.

Programming Problem 2: Maximal Value

- Complete the function msi(x)
- You are allowed to use the sum function
- You might find the following from the lecture useful:

```
1 >>> x = [1,2,3,-2]
2 >>> sum(x[1:3])
3 5
```

Problem 3: Caeser Cipher

The Caeser Cipher (CC) is a means of encoding a message. Please visit
`https://en.wikipedia.org/wiki/Caesar_cipher`
to read more about it (though it's not as complete as one would hope). We have decided to encode messages for this class using the CC, but with a slight modification. We'll only use the lower case alphabet, and we'll include space as an actual symbol. If you look at the character after 'z' in ASCII, you'll see the left brace "{".

```
>>> for i in range(ord('x'), ord('x') + 4):
...     print(f"{i} is {chr(i)} in ASCII")
...
'120 is x in ASCII'
'121 is y in ASCII'
'122 is z in ASCII'
'123 is { in ASCII'
>>>
```

We are including "{" in our alphabet after z as the symbol for space. You'll write two functions, encode(msg, shift) and decode(msg, shift) that encode a message and decode the message using shift as discussed in the wikipedia page, except our alphabet is one symbol larger. The following code:

```
data = ["abc xyz","the cat", "i love ctwohundred"]
for i,j in enumerate(data,start=2):
    print(f"original msg {j}")
    print(f"encoded  msg {encode(j,i)}")
    print(f"decoded  msg {decode(encode(j,i),i)}")

secret_msg = encode("the quick brown fox jumps over the lazy dog", 24)
print(secret_msg)
print(decode(secret_msg,24))
```

produces:

```
original msg abc xyz
encoded  msg cdebz{a
decoded  msg abc xyz
original msg the cat
encoded  msg wkhcfdw
decoded  msg the cat
original msg i love ctwohundred
encoded  msg mdpszidgx{slyrhvih
decoded  msg i love ctwohundred
```

```
10 qebxnrf{hxzoltkxcluxgrjmpxlsboxqebxiywvxald
11 the quick brown fox jumps over the lazy dog
```

Our output is obviously different from the wikipedia example, since we're expanding our alphabet to consider space a letter and including that in the shift.

Deliverables Programming Problem 3

- Complete the functions.
- You can use any normal string function that is associated with the class.

Problem 4: Entropy

We'll be working with finite probabilities. We can think of probabilities as a list of numbers $p_0, p_1, \ldots, p_n$ such that

$$\begin{aligned} p_i &\geq 0, \quad i = 0, 1, \ldots, n && (4)\\ 1 &= p_0 + p_1 + p_2 + \cdots + p_n && (5)\\ &= \sum_{i=0}^{n} p_i && (6) \end{aligned}$$

In this problem, we will be calculating what's called *Entropy*. Entropy is used in ML, AI, vision, finance, *etc.* Entropy is a measure over a probability distribution. Let $P = \{p_1, p_2, \ldots, p_n\}$ be a finite probability distribution over n objects. Entropy $\mathcal{H}$ defined as:

$$\mathcal{H}(P) = -\sum_{i=1}^{n} p_i \log(p_i) \quad (7)$$

Functioning like the mean, this single number that describes how random the data is. If it's 0, then the data isn't random. It means that one object has probability 1 and the remaining zero. If there are n objects and each one is equally probable, then the entropy is maximal: $\log(n)$. How might this be useful? Decisions are "better" the less entropy there is.

We can make a list (we'll assume the items are of the same type and immutable) into a probability. For example, consider a list `x = ["a","b","a","c","c","a"]`.

1. gather the items uniquely (*hint: dictionary*)
2. count each time the item occurs
3. create a new list of the `count/len(dictionary)`

The list of probabilities would be y = [3/6,1/6,2/6] (a's probability is 3/6, b's probability is 1/6 and c's probability is 2/6). Observe that if you add up these numbers, they will sum to one. Now that we have the probabilities, we can determine the entropy:

$$\begin{aligned} \mathcal{H}([.5, .17, .33]) &= -(\frac{3}{6}\log_2(3/6) + \frac{1}{6}\log(1/6) + \frac{2}{6}\log_2(2/6)) && (8)\\ &= -(.5(-1.0) + 0.17(-2.58) + .33(-1.58)) && (9)\\ &= 1.46 && (10) \end{aligned}$$

Because of continuity arguments we treat $\log_2(0) = 0$. Python's math module will correctly state this math error, so you'll **have to simply add 0 if the probability is 0**.

Deliverables Problem 4

- Use $\log_2$
- Take a non-empty list of objects and calculate the entropy.
- You **cannot** use the `count` function.
- Complete the functions for this problem.
- Hint: The makeProbability function is based on equation (3). Use the makeProbability function in your entropy() function.
- Round the output of entropy to two decimal places.

Problem 5: Run of 1s

For a financial instrument like stock, we want to buy and sell at the lowest and highest values, respectively. By observing trends, better decisions about buying and selling can be made. A "trend" is when when a sequence of values is **monotonic**. A sequence of values $v_0, v_1, v_2, \ldots, v_n$ is increasingly monotonic if

$$v_0 \leq v_1 \leq \cdots \leq v_n$$

The increasing monotonicity allows for selling at a higher price. Conversely, there is decreasing monotonicity. In this problem we'll be writing code to look for increasing monotonicity, but simplifying the problem by using only zero or one. In this problem, you'll take a list of 0s and 1s as an input. Your program will output the *longest* sequential run of 1s (no 0s encountered).

Output function.py

```
L([1, 0, 1, 0, 1, 0])
1

L([0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0])
2

L([1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1])
4
```

Deliverables Problem 5

- Complete the function.
- Use any method of looping you'd like, but there exists a simple version that uses a single bounded loop

Problem 6: Nine of Cattails

It's known that if you add the digits of any number, and that it eventually equals nine, that the original number is divisible by nine. Here is an example:

$$39789 \quad \rightarrow \quad 3+9+7+8+9 = 36 \rightarrow 3+6 = 9$$

It doesn't matter the order, *viz.,*

$$3+((9+7)+8)+9$$
$$3+((16)+8)+9$$
$$3+(7+8)+9$$
$$3+15+9$$
$$18+9$$
$$27 \rightarrow 9$$

We are left with only one digit which is 9 and hence the number 39789 is divisible by 9 (Infact, 9 * 4421 = 39789).

So we can see that the number 39789 is indeed divisible by 9 by following our method. Write a program that determines if a number is divisible by nine by using this method. You *cannot* simply divide by 9 and return True! You must continually add the numbers until a single digit is left; that digit will either be 9 or something else.

Caution: Remember that 'sum' is actually a built-in function in Python so we suggest that you store the results of addition in a variable named other than 'sum' to prevent errors or failures during testing.

sum = a + b + c (this is not a good practise because 'sum' is a reserved keyword in Python), rather
my_sum = a + b + c (this is safe to do)

Deliverables Problem 6

- Complete the functions **following** the algorithm described above.
- You can neither use the % sign in the program nor divide directly by 9.

Problem 7: Tiling

In this problem, you're given a collection of tiles as numbers, for example, [1,2,3]. You're given a space to fit the tiles (linearly), for example 6. Your function gives all the possible patterns where the tiles add exactly to the fit. For example,

```
1   n = 6
2   v = [1,2,3]
3   print(tiles(n,v,[[i] for i in v]))
4   for i in tiles(n,v,[[i] for i in v]):
5         print(sum(i), end="")
```

produces:

```
1 [[3, 3], [3, 2, 1], [2, 3, 1], [3, 1, 2], [2, 2, 2], [1, 3, 2], [2, 1,↩
      3],
2  [1, 2, 3], [3, 1, 1, 1], [2, 2, 1, 1], [1, 3, 1, 1], [2, 1, 2, 1],
3  [1, 2, 2, 1], [1, 1, 3, 1], [2, 1, 1, 2], [1, 2, 1, 2], [1, 1, 2, 2],
4  [1, 1, 1, 3], [2, 1, 1, 1, 1], [1, 2, 1, 1, 1], [1, 1, 2, 1, 1],
5  [1, 1, 1, 2, 1], [1, 1, 1, 1, 2], [1, 1, 1, 1, 1, 1]]
6 666666666666666666666666
```

All the possible configurations and all equal 6. For numbers [1,2] and size = 4, we have:

```
1 [[2, 2], [2, 1, 1], [1, 2, 1], [1, 1, 2], [1, 1, 1, 1]]
2 44444
```

The function tiles takes n (the total size), v (the different numbers), and the starter tiles. When approaching this problem, you should inspect the output–it is helpful wrt what the recursion does.

Programming Problem 2: Tiling

- Complete the function
- You are free to use any form of looping, but recursion with comprehension seems to be the simplest approach

Problem 9: Models Least Squares

Ordinary Linear Regression (OLS) is the oldest AI technique to make sense of data. The task is to determine the best linear relationship between two groups of data, usually numerical. In this example, we determine the best linear equation for pairs of numbers. A model is a characterization of something. A mathematical model is a formula that describes a relationship among values. The simplest mathematical formula is a linear equation in 2D:

$$0 \quad = \quad Ax + By + C$$

We usually rewrite the equation above as:

$$y \quad = \quad mx + b$$

This means we can determine any value of y by using x. The relationship is **linear** because we're only using $mx + b$, not for example, x^2. Least squares, as we've discussed in lecture is several hundred years old, but remains the most popular modeling for simple relationships. We are given a set of pairs:

$$data \quad = \quad \{(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)\}$$

and want to build the "best" function $f(x) = \hat{m}x + \hat{b}$. where the $\hat{}$ (called hat) means we're approximating a value (we don't know the actual m, b. Without going through the math, there is now a simple procotol to build $\hat{m}, \hat{b}$:

$$\begin{aligned}
data &= \{(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)\} \\
xy_p &= \sum_{i=0}^{n} x_i y_i \\
x_s &= \sum_{i=0}^{n} x_i \\
y_s &= \sum_{i=0}^{n} y_i \\
x_{sq} &= \sum_{i=0}^{n} x_i^2 \\
\\
S_{xy} &= xy_p - \frac{x_s y_s}{n} \\
S_{xx} &= x_{sq} - \frac{x_s^2}{n} \\
\hat{m} &= \frac{S_{xy}}{S_{xx}} \\
\hat{b} &= \frac{y_s - \hat{m}x_s}{n} \\
f(x) &= \hat{m}x + \hat{b}
\end{aligned}$$

To find the quality of the model, we calculate the so-called $0 \leq R^2 \leq 1$ value. Towards unity, the model is a better predictor–at one, it has no error. Toward zero, it is a worse predictor. We need

$M Payroll	Wins
209	89
139	74
101	86
74	74
67	68
49	67
119	97
98	92

Table 1: Payroll/Wins for eight years.

two values: the total sum of squares SST and the sum of square error SSE. We'll write this as typically seen in Statistics texts:

$$\text{SST} = \sum y^2 - \frac{(\sum y)^2}{n} \tag{11}$$

$$\text{SSE} = \sum y^2 - b\sum y - m\sum xy \tag{12}$$

$$R^2 = \frac{\text{SST} - \text{SSE}}{\text{SST}} \tag{13}$$

While this seems like a lot of loops, using Python comprehension and lambda, we can make quick work of this. Observe each summation is the same–the only thing changing is f. We could write a function that takes the data and function returning the value–here are a couple for y_s and x_{sq}.

```
...
def r(data,f):
        return sum([f(x,y) for x,y in data])
...
y_s = r(data,lambda x,y:y)
x_sq = r(data,lambda x,y:x**2)
...
```

Assume you're working in sports analytics and have the payroll for the team for the last eight years with the number of wins. Table 1 shows the data. Here's a run on some data:

```
m_hat, b_hat, R_sq  = std_linear_regression(data6)
print(m_hat,b_hat, R_sq)
plt.plot([x for x,_ in data6],[y for _,y in data6],'ro')
plt.plot([x for x,_ in data6],[m_hat*x + b_hat for x,_ in data6],'b')
plt.xlabel("$M Payroll")
plt.ylabel("Season Wins")
plt.title(f"Least Squares: m = {m_hat}, b = {b_hat}, R^2 = {R_sq} ")
plt.ylabel("Y")
```

```
9 plt.show()
```

with output:

```
1 0.125 67.5 0.298
```

and graphic:

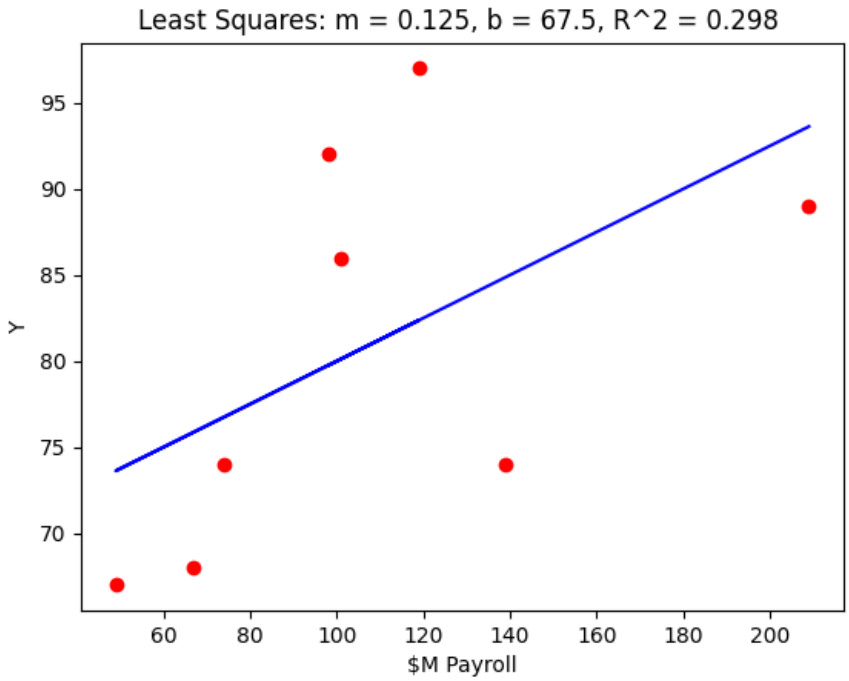

Figure 1: Least squares function (blue) and data (red). The least-squares regression models the linear relationship between the data points. Observe the R^2 value.

Note: For this problem, we have already given the code under (__name__ == __main__) to plot the graph as shown above. After completing this problem, you can un-comment the code to draw the graph. Remember to comment it back before submitting to the Autograder. The Autograder can not draw graphical plots in the web bowser, and thereofre your submission attempt may fail with errors even though the code is correct, so please comment the code related to the plot before making final submission.

Interestingly, the numpy module has has regression as a function. We demonstrate it showing it agrees with our code. Please run this, since the next homework will use polyfit.

```
1 import numpy as np
2
3 data = [(209,89),(139,74),(101,86),(74,74),(67,68),
4         (49,67),(119,97),(98,92)]
5 x = [x for x,_ in data]
6 y = [y for _,y in data]
```

```
7
8  slope, intercept = np.polyfit(x,y,1)
9
10 print(slope,intercept)
```

produces

```
1  0.1250140908578519 67.49849227820988
```

Deliverables Programming Problem 8

- You must create a list structure to hold the data presented in Table 1.
- Complete the function–you are encouraged to use comprehension and lambda, but are free to implement this as you wish
- You cannot use any existing tools for regression–we'll be using those later
- Round the final values of $\hat{m}$, $\hat{b}$, and R^2 to three decimal places

1 Programming Partners

drmiguth@iu.edu, dematt@iu.edu
shkami@iu.edu, sasubb@iu.edu
rkoston@iu.edu, bsati@iu.edu
asgurram@iu.edu, mzuhair@iu.edu
pikhatri@iu.edu, roberjuv@iu.edu
mahchaud@iu.edu, tlichten@iu.edu
rbernfel@iu.edu, ydpatil@iu.edu
heissler@iu.edu, cpmathew@iu.edu
evarcarm@iu.edu, dmonroyh@iu.edu
clarkina@iu.edu, omwells@iu.edu
abrking@iu.edu, martibr@iu.edu
ckbarts@iu.edu, westje@iu.edu
laigrant@iu.edu, marsh1@iu.edu
mgbekele@iu.edu, braxluns@iu.edu
owencape@iu.edu, rpolu@iu.edu
adj18@iu.edu, nmenne@iu.edu
mailic@iu.edu, jw363@iu.edu
jagoodal@iu.edu, dommonte@iu.edu
rdotsu@iu.edu, mmaplet@iu.edu
kkunshet@iu.edu, mkreddy@iu.edu
cgcheane@iu.edu, adimaran@iu.edu
ljhorman@iu.edu, jacloise@iu.edu
chfurlow@iu.edu, cjlomax@iu.edu
nwconn@iu.edu, samyadav@iu.edu
mfbah@iu.edu, mjo4@iu.edu
zahaider@iu.edu, roldenbu@iu.edu
grhamlin@iu.edu, ozopich@iu.edu
ahsans@iu.edu, slopezve@iu.edu
mjfarrin@iu.edu, dajshaw@iu.edu
adatwani@iu.edu, fasrasoo@iu.edu
rycostin@iu.edu, msharsh@iu.edu
ebederma@iu.edu, esspatel@iu.edu
agahlau@iu.edu, bogawa@iu.edu
heoji@iu.edu, aramanu@iu.edu
lafly@iu.edu, mahonm@iu.edu
sdondeti@iu.edu, navann@iu.edu
blstcoop@iu.edu, corawang@iu.edu
drussow@iu.edu, ctsvetan@iu.edu
bdgonzal@iu.edu, olsoncs@iu.edu

nbriere@iu.edu, sstickl@iu.edu
ndodoh@iu.edu, adivaidy@iu.edu
jejea@iu.edu, angzamor@iu.edu
tbashee@iu.edu, liaobrie@iu.edu
etferba@iu.edu, muksingh@iu.edu
natkwon@iu.edu, clushell@iu.edu
pchitwoo@iu.edu, alzemlya@iu.edu
jaccryst@iu.edu, ncw1@iu.edu
ojallow@iu.edu, ansjmeht@iu.edu
vicbarre@iu.edu, elwesley@iu.edu
jackchap@iu.edu, ainlukas@iu.edu
egilmour@iu.edu, sunathan@iu.edu
aare@iu.edu, stewrach@iu.edu
iblay@iu.edu, tkthang@iu.edu
sergonza@iu.edu, seuryu@iu.edu
kadlakha@iu.edu, vnathany@iu.edu
enkarl@iu.edu, manpate@iu.edu
zkabacin@iu.edu, jewach@iu.edu
alurs@iu.edu, prmundra@iu.edu
maboncos@iu.edu, kalerobi@iu.edu
kevgross@iu.edu, jakepars@iu.edu
adekolaw@iu.edu, olesmith@iu.edu
ramgowda@iu.edu, mt87@iu.edu
iyersan@iu.edu, tersacks@iu.edu
salvchri@iu.edu, vpotluri@iu.edu
ag38@iu.edu, jwubelho@iu.edu
nicdelg@iu.edu, ndmyer@iu.edu
mgoenka@iu.edu, sholove@iu.edu
twchase@iu.edu, cjrodenb@iu.edu
jjithesh@iu.edu, mloko@iu.edu
bcarpine@iu.edu, negia@iu.edu
keswa@iu.edu, aposton@iu.edu
ahmealsh@iu.edu, sumuth@iu.edu
sabyrap@iu.edu, lvumpa@iu.edu
ghargro@iu.edu, nmeely@iu.edu
dbarbari@iu.edu, atsarver@iu.edu
natkling@iu.edu, anrajput@iu.edu
jb108@iu.edu, tanilitt@iu.edu
nfarzane@iu.edu, rvarrier@iu.edu
jkittur@iu.edu, antonegr@iu.edu
sydforem@iu.edu, aluckhau@iu.edu

armstan@iu.edu, sahasrir@iu.edu
michdool@iu.edu, mitcal@iu.edu
dajuhnke@iu.edu, jasreese@iu.edu
jbosio@iu.edu, dyllopez@iu.edu
mkulekci@iu.edu, bobbyun@iu.edu
nikdevra@iu.edu, isvoor@iu.edu
mdevara@iu.edu, froy@iu.edu
smhamrah@iu.edu, aidsteph@iu.edu
felkhatt@iu.edu, amarynow@iu.edu
coobratt@iu.edu, aamuse@iu.edu
nimjain@iu.edu, mawlorch@iu.edu
joglickm@iu.edu, kalovert@iu.edu
ehchoo@iu.edu, jskains@iu.edu
ealfalah@iu.edu, shwanara@iu.edu
clickc@iu.edu, hshyama@iu.edu
aarepal@iu.edu, mjseymou@iu.edu
eabilius@iu.edu, ksabloak@iu.edu
jergalan@iu.edu, rvenigal@iu.edu
florech@iu.edu, robreese@iu.edu
mrb9@iu.edu, kpazdur@iu.edu
moazimi@iu.edu, rnalluri@iu.edu
am280@iu.edu, wl52@iu.edu
nafike@iu.edu, rasahu@iu.edu
crigonza@iu.edu, prschulz@iu.edu
mbenites@iu.edu, athason@iu.edu
edeitchl@iu.edu, bawsung@iu.edu
kbussel@iu.edu, youngct@iu.edu
reclay@iu.edu, whitedyl@iu.edu
damaatki@iu.edu, bgtruong@iu.edu
evbutts@iu.edu, ssp4@iu.edu
adhimat@iu.edu, anupand@iu.edu
jigarnes@iu.edu, mclindbe@iu.edu
jbriar@iu.edu, cpmann@iu.edu
yc127@iu.edu, anryoun@iu.edu
ajafowl@iu.edu, cmolaski@iu.edu
ldeveau@iu.edu, jiajtang@iu.edu
mabana@iu.edu, swaschow@iu.edu
ssdatla@iu.edu, ijrussel@iu.edu
radutta@iu.edu, zs15@iu.edu
tycarmo@iu.edu, mszach@iu.edu
srkagema@iu.edu, nshnoble@iu.edu

cjo3@iu.edu, irodeman@iu.edu
bahls@iu.edu, tnerkar@iu.edu
herncr@iu.edu, ibshaw@iu.edu
talfares@iu.edu, fsafianu@iu.edu
nitiarun@iu.edu, sxanders@iu.edu
boydmad@iu.edu, aralover@iu.edu
abondada@iu.edu, adenyu@iu.edu
langarbe@iu.edu, hsh1@iu.edu
jk314@iu.edu, zz67@iu.edu
idasilva@iu.edu, ishyadav@iu.edu
benchart@iu.edu, smitaa@iu.edu
carthugh@iu.edu, maymat@iu.edu
byersjac@iu.edu, hmahapat@iu.edu
aabdulw@iu.edu, jrwaren@iu.edu
mugabr@iu.edu, nealsing@iu.edu
lddial@iu.edu, everma@iu.edu
akoduru@iu.edu, ozsachs@iu.edu
hdjour@iu.edu, jl367@iu.edu
jhl14@iu.edu, thebmill@iu.edu
okirchen@iu.edu, hailreid@iu.edu
ttfender@iu.edu, risvphil@iu.edu
ajkoutsa@iu.edu, joevu@iu.edu
cammlanc@iu.edu, djtheodo@iu.edu
bgelard@iu.edu, ssanjee@iu.edu
dayinla@iu.edu, cmellgre@iu.edu
nifredri@iu.edu, jawegman@iu.edu
cdasari@iu.edu, isundara@iu.edu
bllfost@iu.edu, anangla@iu.edu
jdkakare@iu.edu, kokulski@iu.edu
bedani@iu.edu, marcmarq@iu.edu
cedrchen@iu.edu, jakpatel@iu.edu
meiyhe@iu.edu, kr15@iu.edu
vkabra@iu.edu, conaitis@iu.edu
mthinman@iu.edu, vivitran@iu.edu
jaeblank@iu.edu, camoverm@iu.edu
zalsamri@iu.edu, kenmoses@iu.edu
jkuk@iu.edu, soyedele@iu.edu
gl25@iu.edu, vpremku@iu.edu
blbinder@iu.edu, kiloscot@iu.edu
rgyasini@iu.edu, nguyen17@iu.edu
dgiffor@iu.edu, cs155@iu.edu

pranchan@iu.edu, kevwong@iu.edu
elscheng@iu.edu, sandro@iu.edu
btbasing@iu.edu, isapittm@iu.edu
goldfusl@iu.edu, pjminne@iu.edu
adguhan@iu.edu, cardnorr@iu.edu
gkreutes@iu.edu, jnimmala@iu.edu
dh43@iu.edu, vpottete@iu.edu
alneyadk@iu.edu, rheashah@iu.edu
timolaws@iu.edu, lisalaza@iu.edu
garmenda@iu.edu, lemarc@iu.edu
adodaro@iu.edu, kzerbino@iu.edu
jamcham@iu.edu, zhangsuo@iu.edu
elehaged@iu.edu, aishravi@iu.edu
agotanco@iu.edu, matyun@iu.edu
ygajjar@iu.edu, audscha@iu.edu
eberens@iu.edu, gsackie@iu.edu
bronburk@iu.edu, klohiya@iu.edu
coubenne@iu.edu, jmsequei@iu.edu
xbchrist@iu.edu, rvasude@iu.edu
xmdrew@iu.edu, auskirvi@iu.edu
skalluru@iu.edu, aidpride@iu.edu
skyangel@iu.edu, mmmatias@iu.edu
addfishe@iu.edu, wschrama@iu.edu
ldeel@iu.edu, jjrinald@iu.edu
rogoenka@iu.edu, jopantoj@iu.edu
nantinao@iu.edu, taylbryo@iu.edu
mahichan@iu.edu, davepais@iu.edu
yaljabri@iu.edu, pp22@iu.edu
ethhanna@iu.edu, emasseng@iu.edu
jlaytin@iu.edu, prnemani@iu.edu
skapure@iu.edu, reshetty@iu.edu
banksda@iu.edu, mmalviya@iu.edu
btcunnin@iu.edu, missavin@iu.edu
bthenson@iu.edu, evanmayo@iu.edu
concraft@iu.edu, neenpate@iu.edu
efoltyn@iu.edu, tunnitha@iu.edu
augguy@iu.edu, pmiesch@iu.edu
chjflor@iu.edu, kw125@iu.edu
adagar@iu.edu, drimawi@iu.edu
dakomi@iu.edu, smitfi@iu.edu
tborder@iu.edu, nieywill@iu.edu

keswar@iu.edu, jvmenach@iu.edu
ethdeatr@iu.edu, lydiwang@iu.edu
akeshav@iu.edu, rosanghv@iu.edu
agrief@iu.edu, cmekat@iu.edu
jahjacks@iu.edu, lukthiel@iu.edu